
Translating the solution to a user-friendly end product

Snapchat Masking & Hacking

Zoe J.D.M. Spyrou

Student Number: 452280

Fontys HBO-ICT

Semester 4 OL

Table of Contents

- [Snapchat Masking & Hacking](#)
- [Edit Log](#)
- [Context](#)
- [Objective](#)
- [Scope](#)
- [Research Methodology](#)
- [Vocabulary](#)
- [Process](#)
 - [Design](#)
 - [How the GUI was made](#)
 - [Comparison of Popular Python GUI Libraries](#)
 - [Why a GUI Was Chosen Over a Web Interface](#)
 - [Android Screen Streaming Implementation](#)
 - [Overview](#)
- [Problems Encountered and Solutions](#)
 - [Device Preparation and Manual Commands Used](#)
 - [Final Implementation](#)
 - [Performance Characteristics](#)
- [Summary and Next Steps](#)
- [Sources](#)

Edit Log

Date	Task
19/12/2025	First version

Introduction

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.

Masking is defined as:

- Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
- Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
- Making an account look relatively active (snapscore not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
- Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

Context

The steps taken to create the graphical interface with which the police will be interacting with the VCAM module and sending AI generated images to replace the phone's built in camera; and subsequently the stream that Snapchat uses.

Objective

Create a GUI that is easy to navigate and operate with minimal technical experience.

Scope

Controlled testing on personally owned devices.

Research Methodology

Based on the DOT-Framework, the following methodologies have been utilized:

Method	What/How/Why
Prototyping	Used to design and iterate on the GUI layout. Low- and medium-fidelity prototypes were created to explore usability, workflow, and layout before implementation. This method fits within the workshop category of the DOT framework.
Usability Testing	Informal usability testing was performed during development to identify design flaws and user errors, such as incorrect task order or unclear interaction steps. This aligns with lab-based evaluation methods in the DOT framework.
Technical Experimentation	Iterative technical experiments were conducted to validate screen streaming, device selection, and embedding the phone stream into the GUI. This was necessary to assess feasibility and reliability of the solution.

Vocabulary

FFmpeg

"FFmpeg is a complete, cross-platform solution to record, convert, and stream audio and video. It can decode, encode, transcode, mux, demux, stream, filter, and play media files in any format. It is also highly portable—it compiles and runs in a wide variety of build environments, machine architectures, and configurations like Linux, Mac OS X, Microsoft Windows, etc." (*How to Use FFMpeg in Python (With Examples)*, 2023)

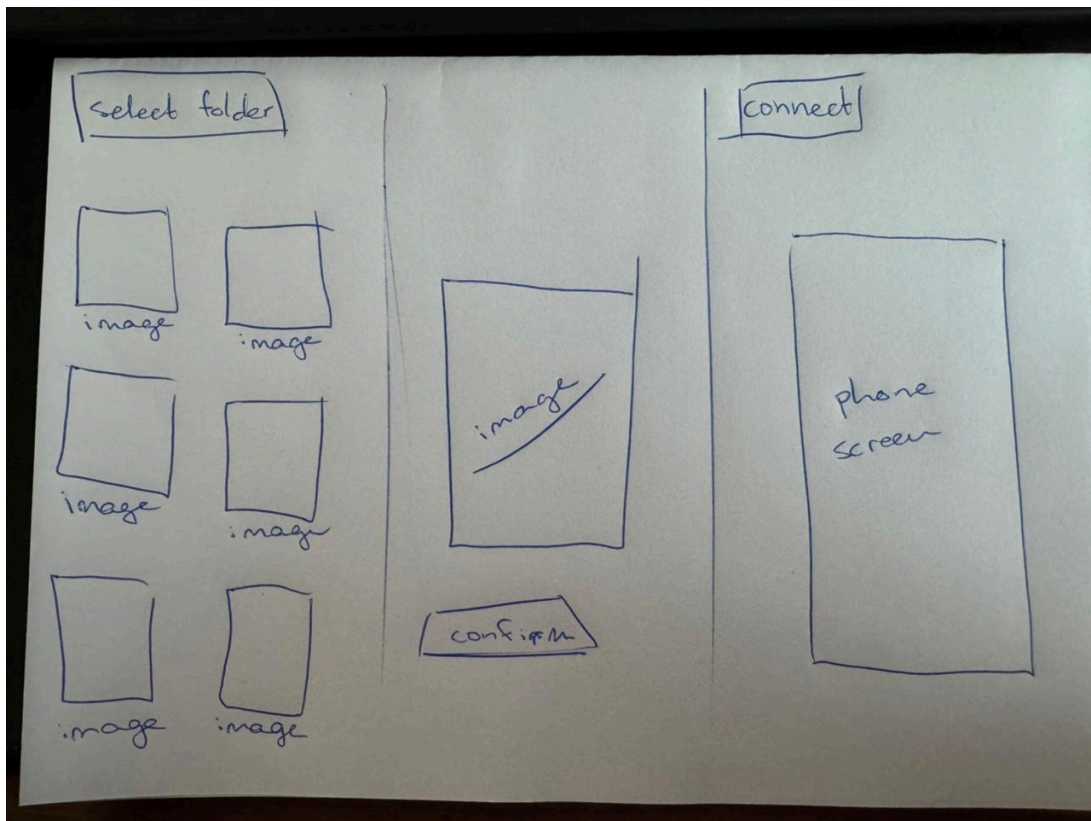
SELinux

"Security-Enhanced Linux (SELinux) is a security architecture for Linux® systems that allows administrators to have more control over who can access the system.", "SELinux defines access controls for the applications, processes, and files on a system. It uses security policies, which are a set of rules that tell SELinux what can or can't be accessed, to enforce the access allowed by a policy." (*What Is SELinux?*, n.d.)

Process

Design

The GUI had to be as simple as possible, with minimal margin of user error allowed/possible. The tasks behind the scene would have to ensure that a simple GUI would suffice.



low_fidelity.jpeg

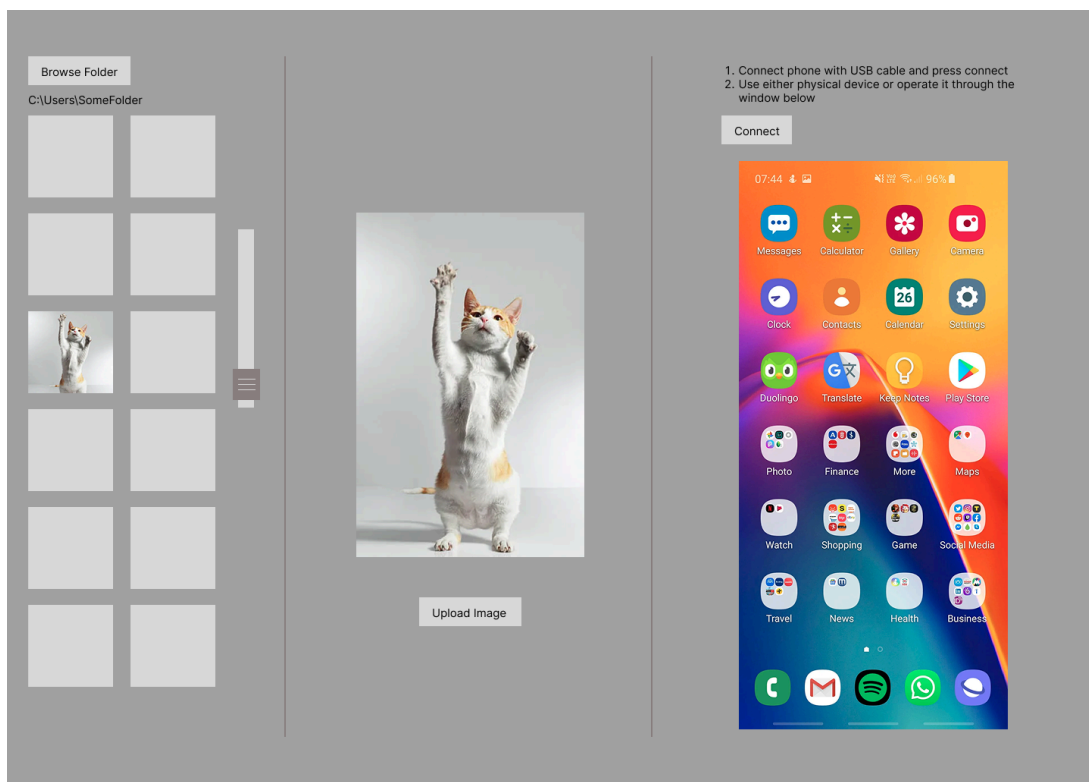


image-102.png

The low and medium fidelity designs conveyed just that, simplicity and clear steps. The police select a folder in which they have generated images of victims, and in the left column, they can view all the images from within that folder, and click on the one they want use for the camera stream. In the middle column, a preview of the selected image is shown, and they click on a button to transfer that image on the phone. Any editing done (resizing, converting the mp4, etc) is done behind the scenes, without the need for user input. In the right column, the police connect with the phone and have a copy of the screen, with which they can interact. This way, they can leave the phone connected aside, and perform all necessary tasks right from within the GUI, instead of using 2 devices.

I had send the second design to the stakeholder, Ronald, who said that the design looks good and intuitive. With his confirmation, I started translating the design into an actual GUI.

The creation of the GUI and choices taken will be explained later in this document.

Initially the design looked as following:

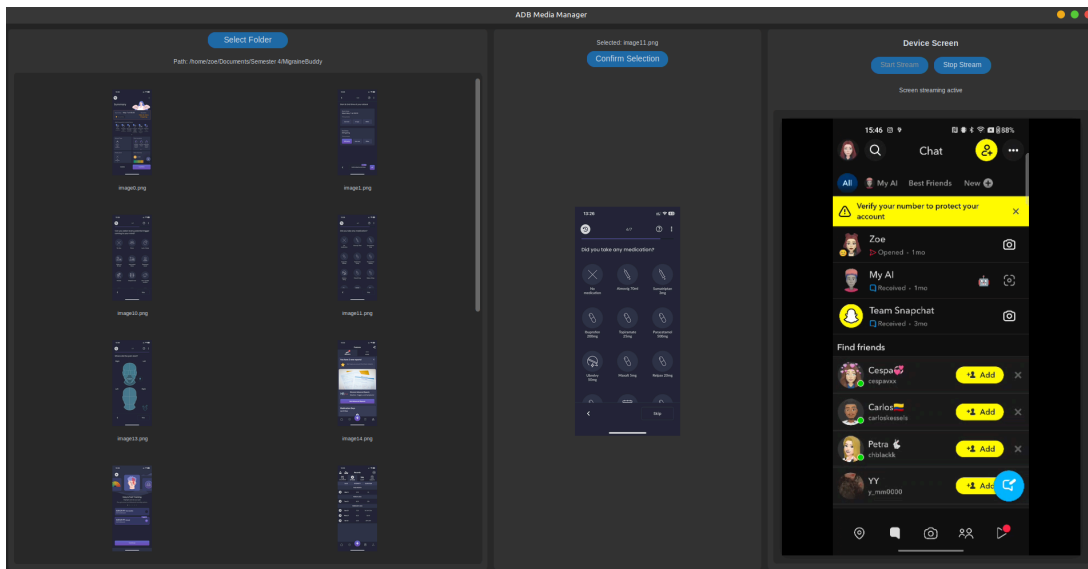


image-99.png

While this design came close to the intended design, with minor tweaks (darker colors for eye comfort), the phone was still not embedded in the actual GUI. It moved along with the rest of the menu and resized responsively, but was still a standalone window. This was not a polished solution, nor good from a software standpoint.

What can also be noted in this version, is that the connect button was replaced with 2 buttons, "start stream" and "stop stream". This was done for the following reason:

Once the initial functionality was built, the design was tested. A folder was opened, an image was selected and to be sent to the phone, when the following error popped up, "No device connected". This was a design flaw. A user reads from left to right, and would not think to connect to the phone prior to attempting an upload. To mitigate this flaw, the app would automatically connect to the Android phone connected to the computer where the menu was running on.

This however opened up a new issue: What if a police officer also connected their personal phone to the computer, to charge it? How would the program respond to that? Which phone would be picked?

This is where the third iteration was born. A menu would first pop up, with a list of devices connected to the computer. The user verifies which phone they will be using, and from there on, operate the GUI as per usual.

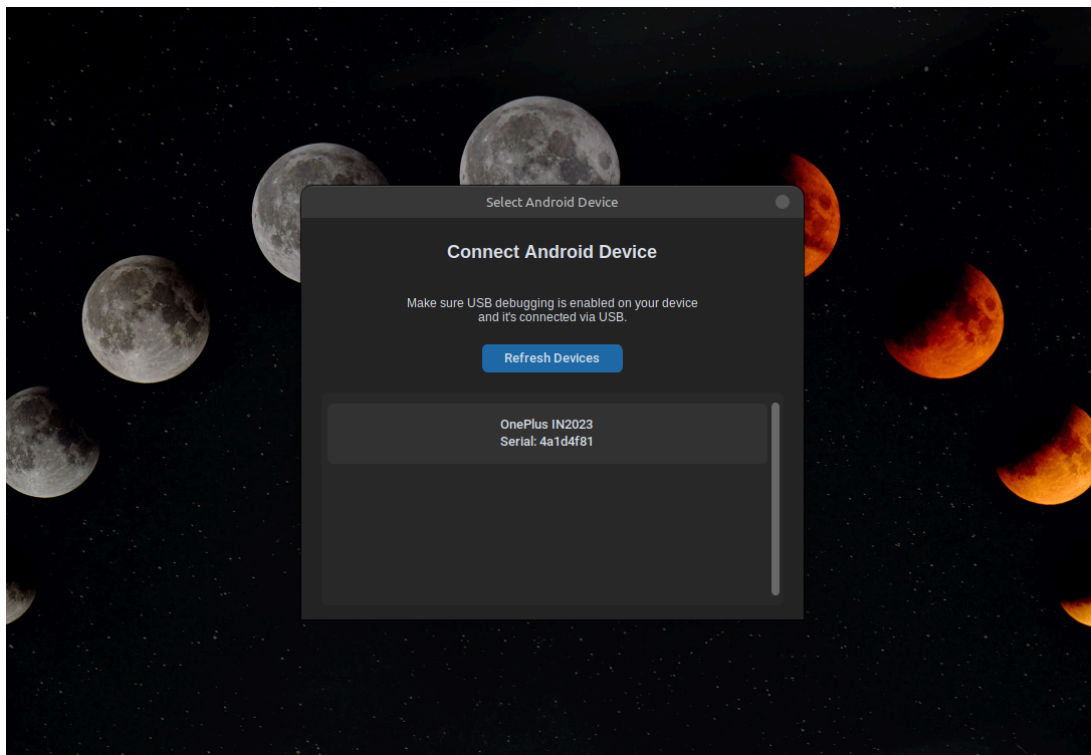


image-103.png

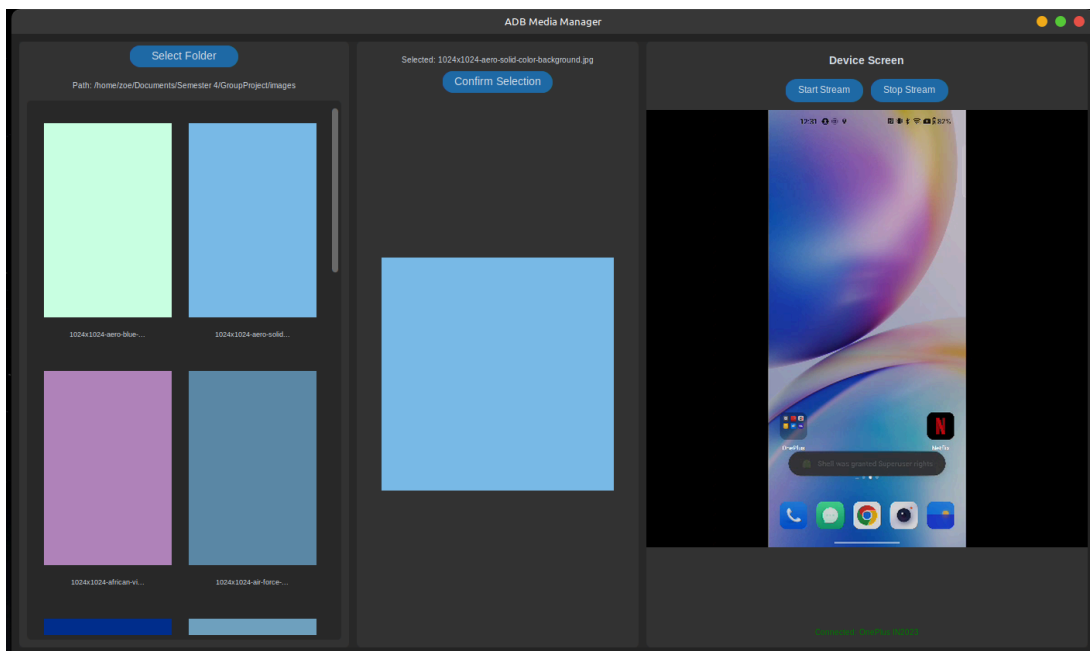


image-104.png

A few more changes were done for this version though, beyond the user experience and "user-proofing" the phone selection menu, by hiding the main menu until a choice was made.

The columns were re-balanced, the left column a bit tighter, making the middle column stand out more. Moreover, the actual phone stream was embedded into the GUI, which was huge upgrade and eliminated the floating window issues. More on the embedding explained further down below.

How the GUI was made

The first step in creating the GUI was choosing the right programming language and interface framework. Python was selected because it allows for fast development, is easy to read, and has a large number of libraries available. This made it well suited for this project, that involved rapid prototyping, image handling, and communication with Android devices.

For the graphical interface, a customized version of Tkinter (*Official Documentation and Tutorial | CustomTkinter*, n.d.) was used, offering the same usability as Tkinter with a modern look. Tkinter (*Tkinter — Python Interface to Tcl/Tk*, n.d.) is included with Python by default, which makes it reliable and easy to deploy without extra dependencies. Other GUI frameworks such as PyQt and Kivy were considered, but they introduce a steeper learning curve and additional complexity that was not necessary for this project, especially given the limited timeframe.

Comparison of Popular Python GUI Libraries

Library	Pros	Cons	Relevance
Tkinter	Built into Python, lightweight, fast to develop	Limited modern styling	Used due to simplicity and speed
PyQt / PySide	Modern look, very flexible	Steep learning curve, licensing concerns	Too complex for project scope
Kivy	Touch-friendly, cross-platform	Less suited for desktop workflows	Not suitable for desktop-focused use

Why a GUI Was Chosen Over a Web Interface

A native desktop GUI was chosen instead of a web interface for practical reasons. While a web application running on localhost could provide similar functionality, a GUI allows for more direct interaction with the operating system and connected devices.

The application requires stable access to USB-connected Android devices, real-time screen streaming, and camera manipulation. Implementing this reliably in a native application is more straightforward than routing everything

through a browser environment. A standalone GUI also reduces setup complexity and avoids browser compatibility issues.

Additionally, the goal was to create a single, self-contained tool that can be launched and used immediately. This aligns better with a desktop application than with a web-based solution that would require running and managing a local server.

Android Screen Streaming Implementation

Overview

Initially, the goal was to embed `scrcpy` directly into the GUI, as it already provides low-latency Android screen streaming. However, `scrcpy` is primarily designed as a standalone tool and relies on X11 on Linux, which makes embedding it into a cross-platform Python GUI difficult. On Windows, `scrcpy` runs in a separate window and does not expose a straightforward way to embed its output, and containerization solutions such as Docker do not resolve this limitation.

For this reason, a custom streaming pipeline using ADB and FFmpeg was implemented instead. The Android `screenrecord` tool outputs an H264 video stream, which is decoded by FFmpeg and displayed inside a Tkinter canvas.

```
# Core streaming command that pipes ADB screenrecord to
FFmpeg

cmd = f"adb -s {self.device.serial} shell screenrecord --
output-format=h264 --size {VIDEO_WIDTH}x{VIDEO_HEIGHT} --
bit-rate 8000000 - | ffmpeg -i - -f rawvideo -pix_fmt rgb24
-LogLevel error pipe:1"

self.ffmpeg_process = subprocess.Popen(
    cmd,
    shell=True,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    bufsize=0
)
```

Without the decoding, the stream is just a whole lot of random characters, as depicted in the following screenshot:

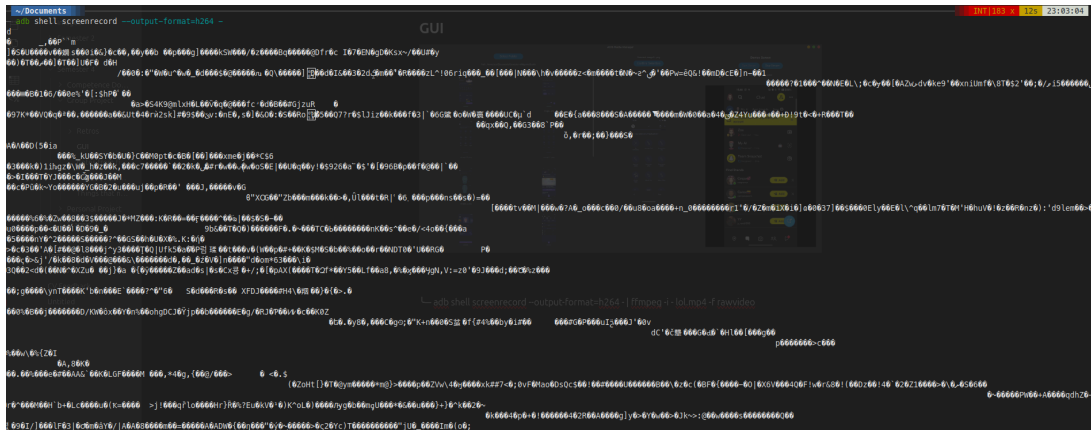


image-100.png

Problems Encountered and Solutions

During implementation, several issues were encountered. Initial attempts resulted in a black canvas, despite the streaming pipeline running without errors. This was caused by a combination of format detection issues, buffering behavior, and incomplete frame reads.

Forcing FFmpeg to treat the input as raw H264 caused decoding problems. Allowing FFmpeg to auto-detect the input format resolved this. Running ADB and FFmpeg as separate processes with manual piping also caused synchronization issues. This was solved by running both commands as a single shell pipeline, allowing the operating system to handle buffering correctly.

FFmpeg outputs frames in chunks rather than complete frames. To handle this, frames were read incrementally until the full frame size was reached. A short buffering delay was also added at startup to allow FFmpeg to collect enough data before decoding.

```
# Read frame in chunks until we have the complete frame

raw = b''
bytes_needed = FRAME_SIZE
max_attempts = 200

while len(raw) < FRAME_SIZE and attempt < max_attempts and
not self.stop_event.is_set():
    chunk_size = min(8192, bytes_needed - len(raw))
```

```

        chunk = self.ffmpeg_process.stdout.read(chunk_size)
    if not chunk:
        break

    raw += chunk
    attempt += 1

# Convert to numpy array and reshape

frame = np.frombuffer(raw, np.uint8)
self.frame = frame.reshape((VIDEO_HEIGHT, VIDEO_WIDTH, 3))

```

Color space mismatches were resolved by switching to the RGB24 pixel format, which is compatible with the Python Imaging Library. On rooted devices, SELinux restrictions sometimes prevented output from screenrecord. Temporarily disabling SELinux was used for testing, but further investigation is required for a production-ready solution.

Finally, UI layout issues were addressed by resizing the video canvas and adjusting the window dimensions to ensure that all controls remained visible.

Device Preparation and Manual Commands Used

During development and debugging, several manual ADB commands were used to verify device behavior and explore permission-related issues. These commands are documented here for reference and reproducibility.

The real-time stream works reliably and is embedded in the GUI, although minor latency is observed and can be optimized further. Files can be pushed from the GUI into non-rooted directories on the phone without issues.

```

def on_image_confirm(self):
    if not self.current_selected_file or not
self.selected_device:
        return

    try:
        filename =
os.path.basename(self.current_selected_file)
        remote_path = f"{self.remote_folder}/{filename}"

        push_file(self.selected_device,

```

```
self.current_selected_file, remote_path)
    self.info_label.configure(text=f"Successfully pushed
{filename} to device")
except Exception as e:
    self.info_label.configure(text=f"Error: {str(e)}")
```

On some rooted devices, SELinux restrictions may prevent certain operations, such as writing directly to system-protected directories. To temporarily disable SELinux enforcement during testing, the following command was used:

```
adb shell su -c "setenforce 0"
```

To manually test whether screen recording and streaming over ADB functioned outside of the GUI, the following pipeline was executed:

```
adb shell screenrecord --output-format=h264 - | ffmpeg -i -
test.mp4 -f rawvideo
```

This confirmed that the stream itself works correctly and can be processed by FFmpeg. However, pushing video files directly to protected directories using ADB has not yet been tested and may fail due to SELinux restrictions. So far, the workaround has been to manually move files into those directories using ZArchiver on the phone. Further testing is required to determine if a fully automated solution via ADB is possible under these constraints.

Final Implementation

The final solution uses a single shell command that pipes the output of `adb screenrecord` directly into FFmpeg, which decodes the stream into raw RGB frames. These frames are converted into images and displayed on the GUI canvas at regular intervals.

This approach resulted in a stable, low-latency stream that works reliably within the GUI. Minor lag is observed and can be optimized further.

Files can be pushed from the GUI into non-rooted directories on the phone without issues. Pushing video files directly into system-protected directories has not yet been fully tested and may be restricted by SELinux. Currently, manual placement using tools such as ZArchiver is used as a workaround. Further testing is required to determine whether automated file placement in these directories is feasible.

Performance Characteristics

- Resolution: 360x480 pixels
- Bitrate: 8 Mbps
- Frame rate: Approximately 30 fps
- Initial buffering time: Approximately 4 seconds
- Steady-state latency: Under 1 second

Summary and Next Steps

What Works Now

This project successfully translated a technically complex solution into a user-friendly graphical interface. Through iterative design, testing, and refinement, the final GUI allows police officers to interact with the VCAM module and Android devices without requiring technical knowledge of the underlying systems.

The current implementation successfully integrates:

- Image selection from folders containing AI-generated images
- Device selection and control via USB using ADB
- Real-time Android screen streaming embedded directly into the GUI

The screen stream works reliably and is displayed inside the application with low latency. Files can be pushed from the GUI into non-rooted directories on the Android device without issues. Overall, the core functionality required to operate the system from a single interface has been achieved.

What Still Needs Testing

While the main functionality is operational, some aspects require further testing and validation.

- The video stream shows minor latency and could benefit from additional performance optimization.
- Pushing video files directly into system-protected or root-level directories has not yet been fully tested. Based on current observations, this may be restricted by SELinux policies.

- The feasibility of automating file placement into these protected directories using ADB, without manual intervention or modifying SELinux settings, remains uncertain.

These limitations do not affect the core functionality but are important for ease of deployment.

Next Steps

The following steps are:

- Optimize the video stream to reduce latency and improve smoothness.
- Implement permission handling directly within the application, rather than relying on manual configuration or external file managers.
- Automatically start and manage the ADB server from within the GUI to reduce setup errors, as noted during the fourth sprint demo.
- Convert images prior to uploading to 10 second long mp4 videos, to replace camera stream, and rename videos to "virtual.mp4".
- Decide and implement a deployment strategy for the final product, comparing options such as Docker containers, PyInstaller, or other packaging methods.
- Make video stream interactive, to more closely mimic `scrcpy`.

Sources

How to Use FFMpeg in Python (with Examples). (2023, August 1).

Bannerbear. <https://www.bannerbear.com/blog/how-to-use-ffmpeg-in-python-with-examples/>

What is SELinux? (n.d.). <https://www.redhat.com/en/topics/linux/what-is-selinux>

Official documentation and tutorial | CustomTkinter. (n.d.).

<https://customtkinter.tomschimansky.com/>

tkinter — Python interface to Tcl/Tk. (n.d.). Python Documentation.

<https://docs.python.org/3/library/tkinter.html>

TurbineThree. (2023, September 19). *Make Tkinter look 10x better in 5 minutes (CustomTkinter)* [Video]. YouTube. https://www.youtube.com/watch?v=Miydkti_QVE

Leng-Yue. (n.d.). *GitHub - leng-yue/py-scrpy-client: An easy to use python scrpy client*. GitHub. <https://github.com/leng-yue/py-scrpy-client>